January 24, 2022

MD-Bench

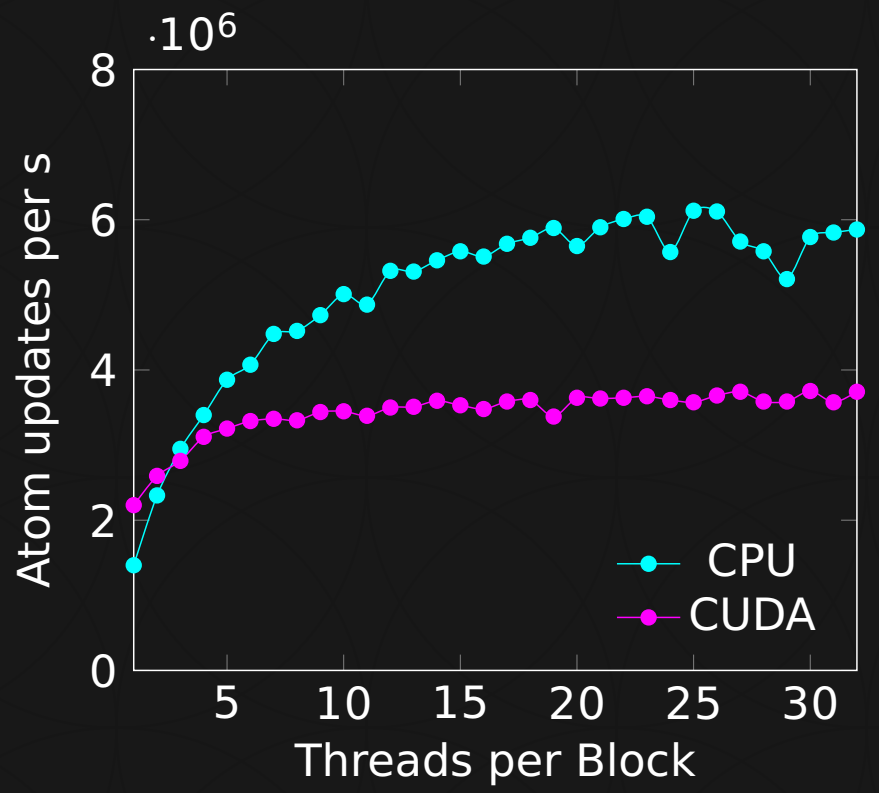
Maximilian Gaul

MuCoSim Seminar

Review

Where we left off I

- Performance



Where we left off II

- Memory throughput

Duration	Size	Throughput	SrcMemType	DstMemType	Name
899.68us	4.1199MB	4.4719GB/s	Pageable	Device	[CUDA memcpy HtoD]
87.904us	1.0000MB	11.109GB/s	Pageable	Device	[CUDA memcpy HtoD]
87.616us	1.0000MB	11.146GB/s	Pageable	Device	[CUDA memcpy HtoD]
87.808us	1.0000MB	11.122GB/s	Pageable	Device	[CUDA memcpy HtoD]
60.640us	703.13KB	11.058GB/s	Pageable	Device	[CUDA memcpy HtoD]
896ns	128B	136.24MB/s	Pageable	Device	[CUDA memcpy HtoD]
864ns	128B	141.29MB/s	Pageable	Device	[CUDA memcpy HtoD]
864ns	128B	141.29MB/s	Pageable	Device	[CUDA memcpy HtoD]
11.534ms	50.000MB	4.2335GB/s	Pageable	Device	[CUDA memcpy HtoD]
45.248us	512.00KB	10.791GB/s	Pageable	Device	[CUDA memcpy HtoD]
19.917ms	-	-	-	-	calcforce() [132]
80.896us	1.0000MB	12.072GB/s	Device	Pageable	[CUDA memcpy DtoH]
80.736us	1.0000MB	12.096GB/s	Device	Pageable	[CUDA memcpy DtoH]
80.480us	1.0000MB	12.134GB/s	Device	Pageable	[CUDA memcpy DtoH]

- atom positions, atom forces, atom type, Lennard-Jones parameters, neighbourlist

Improvements

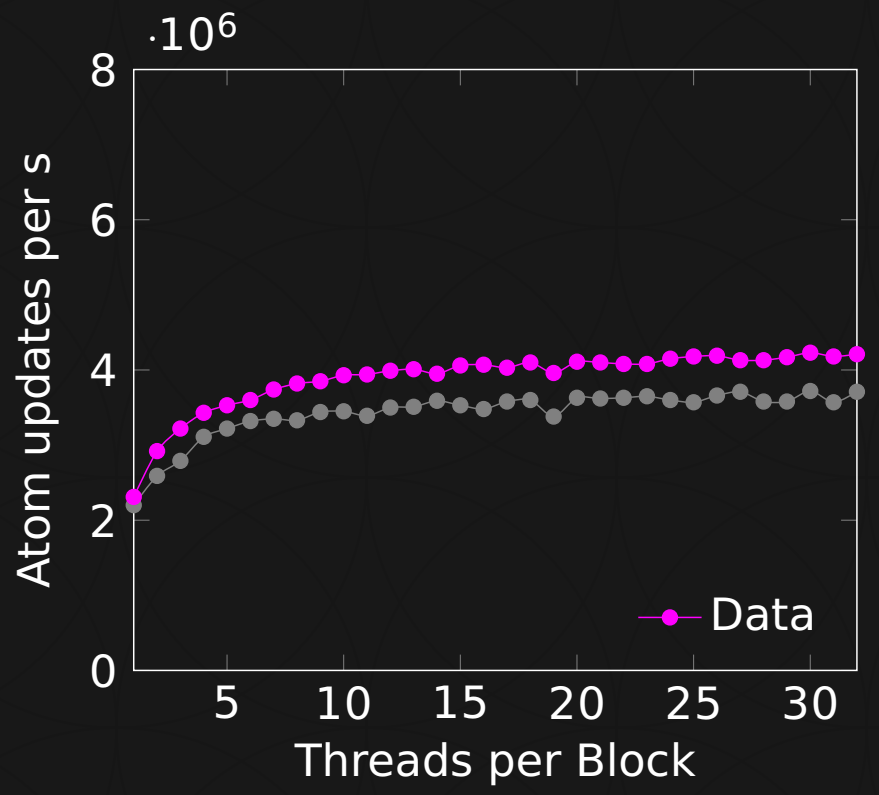
Pinnend memory I

- Memory on the host-side can be pinned via ’cudaMallocHost’ and ’cudaFreeHost’
  - Allocates page-locked memory which can be directly accessed by the GPU and thus results in higher bandwidth
- Pinning the atom-related memory:

Duration	Size	Throughput	SrcMemType	DstMemType	Name
356.25us	4.1199MB	11.293GB/s	Pinned	Device	[CUDA memcpy HtoD]
88.384us	1.0000MB	11.049GB/s	Pinned	Device	[CUDA memcpy HtoD]
87.967us	1.0000MB	11.101GB/s	Pinned	Device	[CUDA memcpy HtoD]
88.128us	1.0000MB	11.081GB/s	Pinned	Device	[CUDA memcpy HtoD]
61.183us	703.13KB	10.960GB/s	Pinned	Device	[CUDA memcpy HtoD]
832ns	128B	146.72MB/s	Pinned	Device	[CUDA memcpy HtoD]
832ns	128B	146.72MB/s	Pinned	Device	[CUDA memcpy HtoD]
832ns	128B	146.72MB/s	Pinned	Device	[CUDA memcpy HtoD]
10.001ms	50.000MB	4.8821GB/s	Pageable	Device	[CUDA memcpy HtoD]
45.056us	512.00KB	10.837GB/s	Pageable	Device	[CUDA memcpy HtoD]
19.900ms	-	-	-	-	calcforce() [292]
80.768us	1.0000MB	12.091GB/s	Device	Pinned	[CUDA memcpy DtoH]
80.959us	1.0000MB	12.062GB/s	Device	Pinned	[CUDA memcpy DtoH]
80.832us	1.0000MB	12.081GB/s	Device	Pinned	[CUDA memcpy DtoH]

Pinnend memory II

- Performance



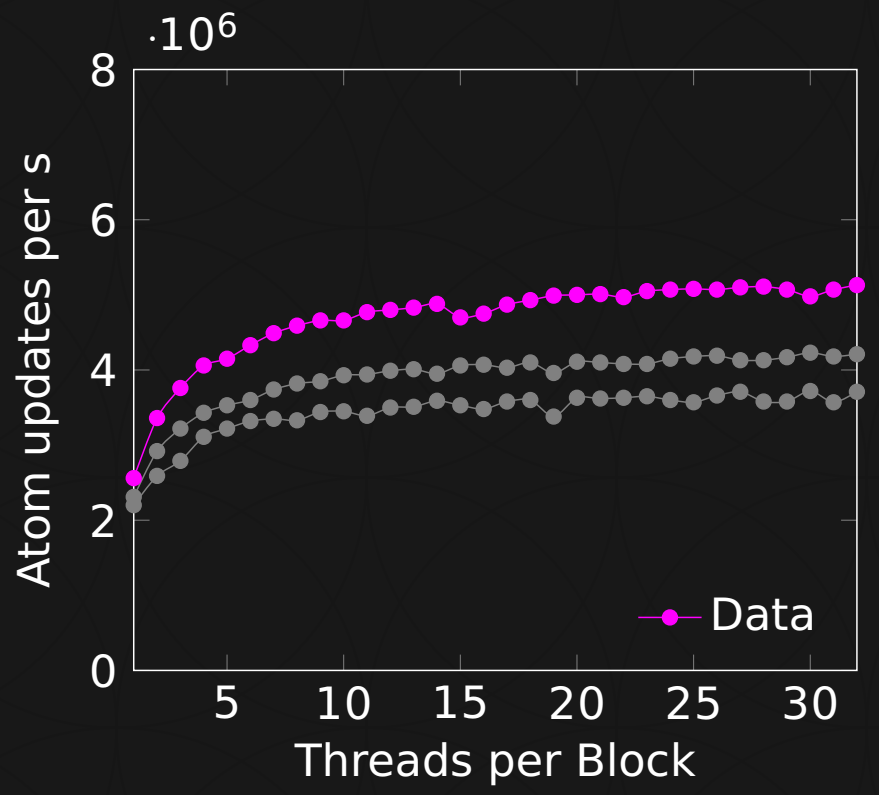
Pinnend memory III

- Pinning all host-side allocated memory (incl. neighbour data)

Duration	Size	Throughput	SrcMemType	DstMemType	Name
355.54us	4.1199MB	11.316GB/s	Pinned	Device	[CUDA memcpy HtoD]
88.069us	1.0000MB	11.089GB/s	Pinned	Device	[CUDA memcpy HtoD]
87.972us	1.0000MB	11.101GB/s	Pinned	Device	[CUDA memcpy HtoD]
87.844us	1.0000MB	11.117GB/s	Pinned	Device	[CUDA memcpy HtoD]
61.187us	703.13KB	10.959GB/s	Pinned	Device	[CUDA memcpy HtoD]
832ns	128B	146.72MB/s	Pinned	Device	[CUDA memcpy HtoD]
833ns	128B	146.54MB/s	Pinned	Device	[CUDA memcpy HtoD]
832ns	128B	146.72MB/s	Pinned	Device	[CUDA memcpy HtoD]
4.2854ms	50.000MB	11.394GB/s	Pinned	Device	[CUDA memcpy HtoD]
45.443us	512.00KB	10.745GB/s	Pinned	Device	[CUDA memcpy HtoD]
19.880ms	-	-	-	-	calcforce() [294]
81.285us	1.0000MB	12.014GB/s	Device	Pinned	[CUDA memcpy DtoH]
80.932us	1.0000MB	12.066GB/s	Device	Pinned	[CUDA memcpy DtoH]
80.868us	1.0000MB	12.076GB/s	Device	Pinned	[CUDA memcpy DtoH]

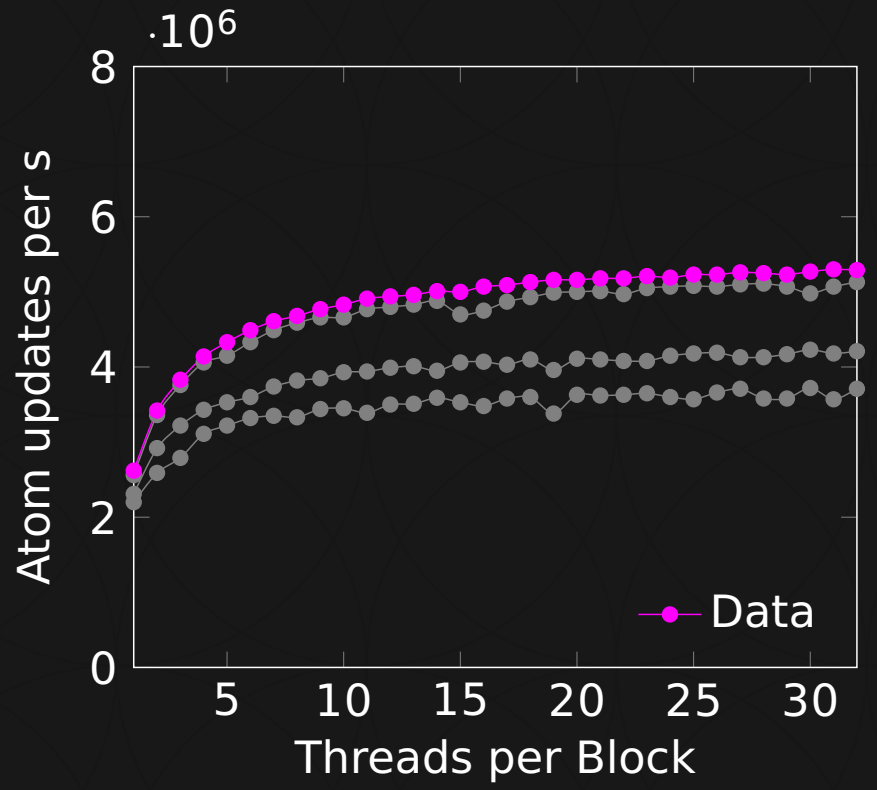
Pinnend memory IV

- Performance



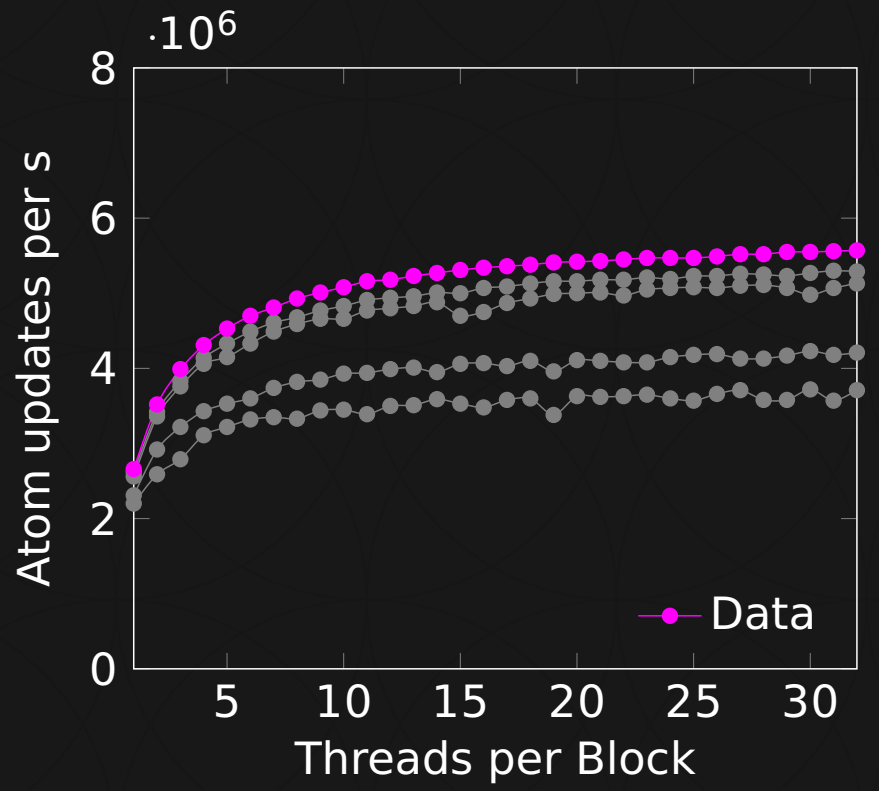
Allocate memory only once

- By allocating memory only once during initialization we can improve performance



Clear force arrays

- Before each calculation, atom forces have to be set to 0. This can also be done with 'cudaMemset'



Avoid repeated copying of parameters I

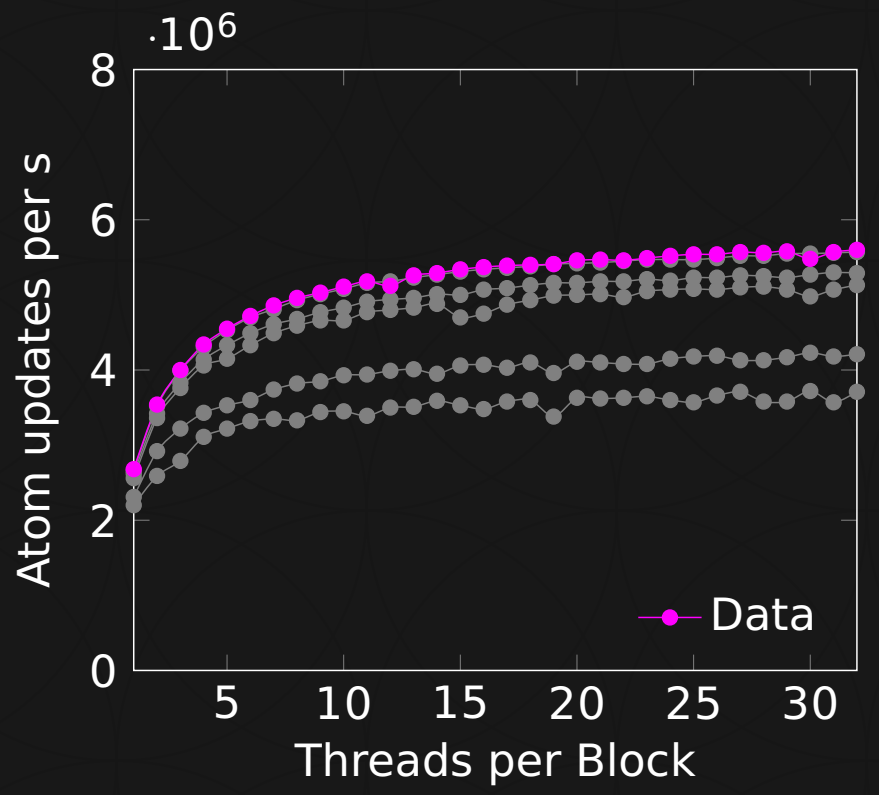
- Lennard-Jones parameters ϵ, σ as well as atom-type and force-threshold need to be copied only once because the are static

Duration	Size	Throughput	SrcMemType	DstMemType	Name
354.67us	4.1199MB	11.344GB/s	Pinned	Device	[CUDA memcpy HtoD]
3.2640us	1.0000MB	299.19GB/s	Device	-	[CUDA memset]
3.2650us	1.0000MB	299.10GB/s	Device	-	[CUDA memset]
2.8800us	1.0000MB	339.08GB/s	Device	-	[CUDA memset]
4.2804ms	50.000MB	11.407GB/s	Pinned	Device	[CUDA memcpy HtoD]
45.058us	512.00KB	10.837GB/s	Pinned	Device	[CUDA memcpy HtoD]
1.5061ms	-	-	-	-	calcforce() [308]
80.771us	1.0000MB	12.091GB/s	Device	Pinned	[CUDA memcpy DtoH]
80.899us	1.0000MB	12.071GB/s	Device	Pinned	[CUDA memcpy DtoH]
80.803us	1.0000MB	12.086GB/s	Device	Pinned	[CUDA memcpy DtoH]

- We are left with atom position, atom forces and neighbour-data

Avoid repeated copying of parameters II

- Performance



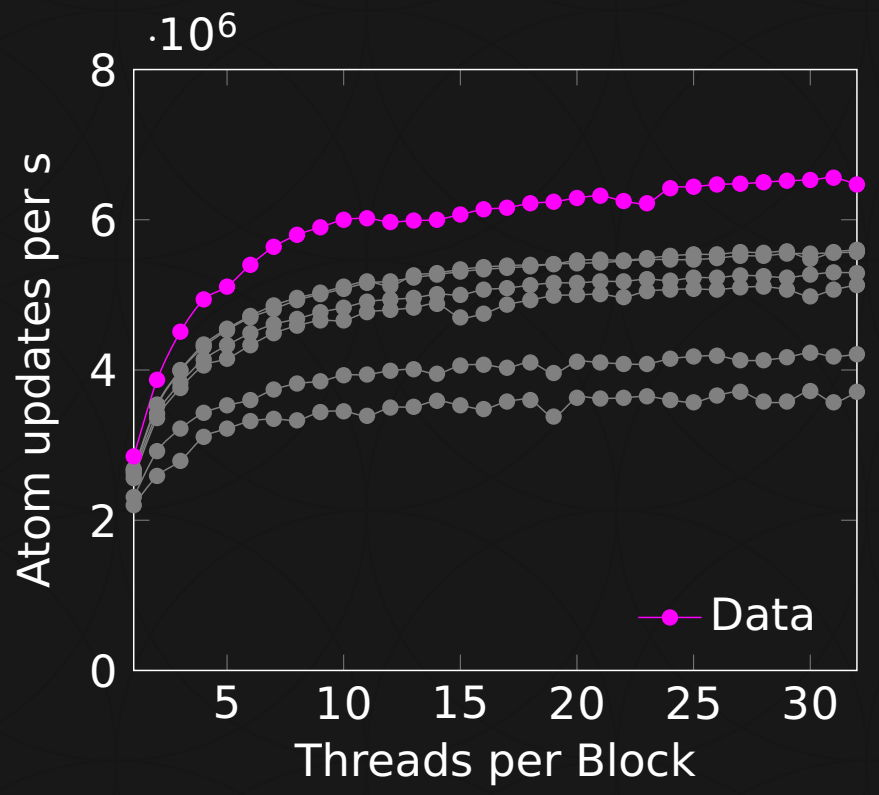
Optimize neighbour list copying I

- Neighbour-lists are updated only every 20 iterations, so there is no need to copy them in every run
- Number of memory transfers can be reduced even further

Duration	Size	Throughput	SrcMemType	DstMemType	Name
354.67us	4.1199MB	11.344GB/s	Pinned	Device	[CUDA memcpy HtoD]
3.3600us	1.0000MB	290.64GB/s	Device	-	[CUDA memset]
3.2960us	1.0000MB	296.29GB/s	Device	-	[CUDA memset]
2.9760us	1.0000MB	328.15GB/s	Device	-	[CUDA memset]
1.4924ms	-	-	-	-	calcforce() [306]
80.740us	1.0000MB	12.095GB/s	Device	Pinned	[CUDA memcpy DtoH]
80.996us	1.0000MB	12.057GB/s	Device	Pinned	[CUDA memcpy DtoH]
80.836us	1.0000MB	12.081GB/s	Device	Pinned	[CUDA memcpy DtoH]

Optimize neighbour list copying II

- Performance



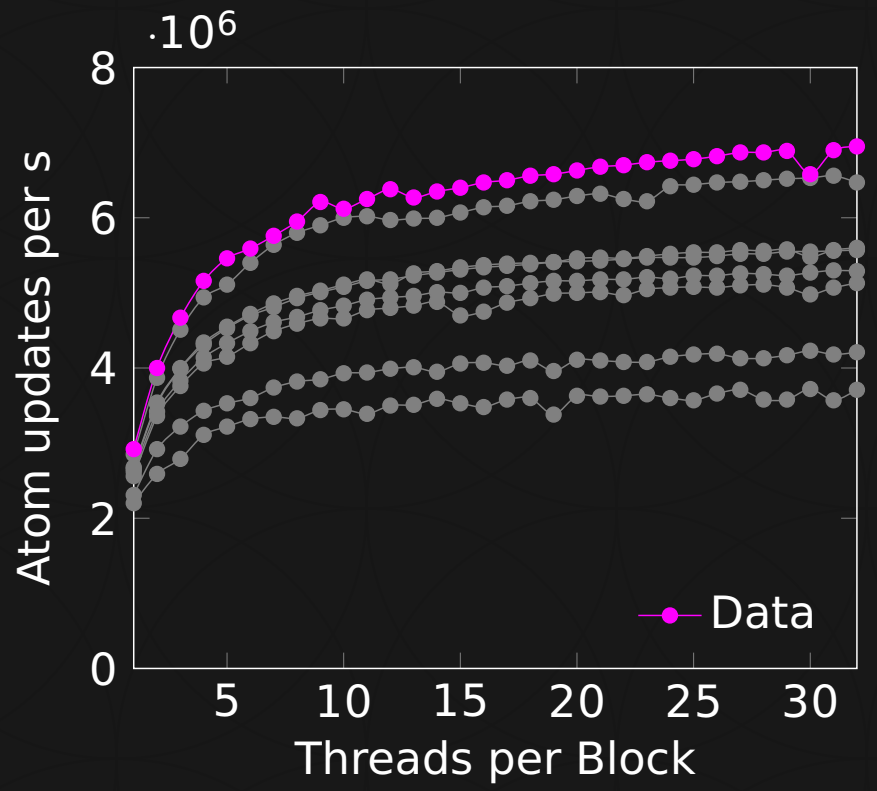
Memory structure of forces to AoS I

- Atom coordinates are already implemented as AoS, do the same for atom forces
  - By doing so, we replace three small by one large memory transfer
- This also effects other parts of the code (e.g. force integration)

Duration	Size	Throughput	SrcMemType	DstMemType	Name
355.01us	4.1199MB	11.333GB/s	Pinned	Device	[CUDA memcpy HtoD]
5.0240us	3.0000MB	583.14GB/s	Device	-	[CUDA memset]
1.5084ms	-	-	-	-	calcforce() [288]
242.34us	3.0000MB	12.089GB/s	Device	Pinned	[CUDA memcpy DtoH]

Memory structure of forces to AoS II

- Performance

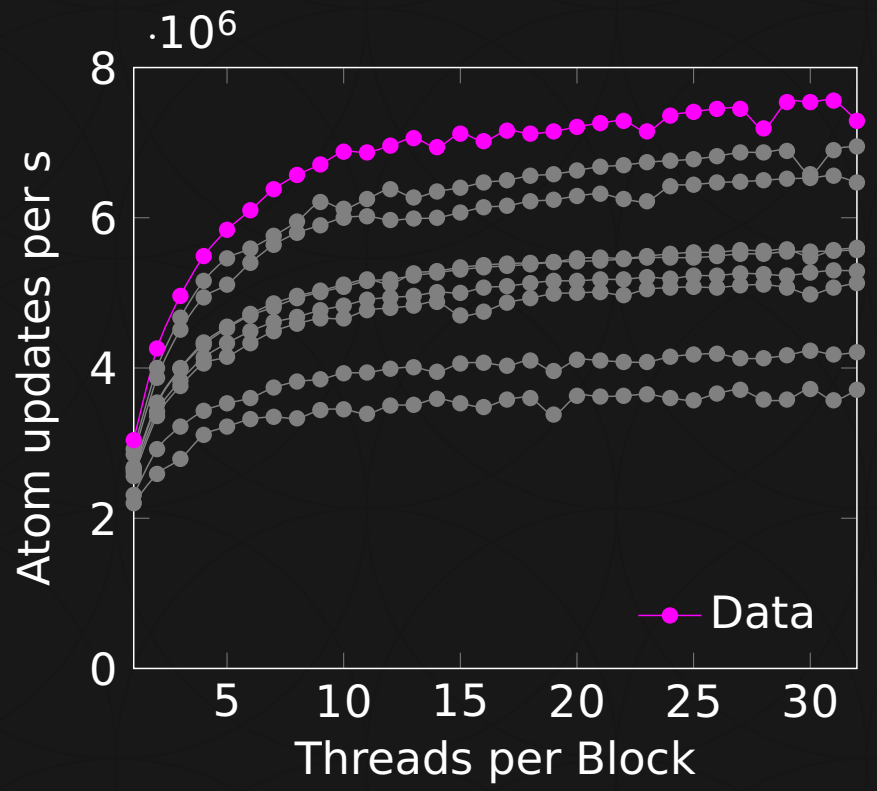


Force integration to AoS & CUDA I

- Change the atom velocity memory layout to AoS as we already did for position and force
- Furthermore, port force integration functions to CUDA kernels and execute them on the GPU
  - Kernels need velocity and position data, so those need to be copied

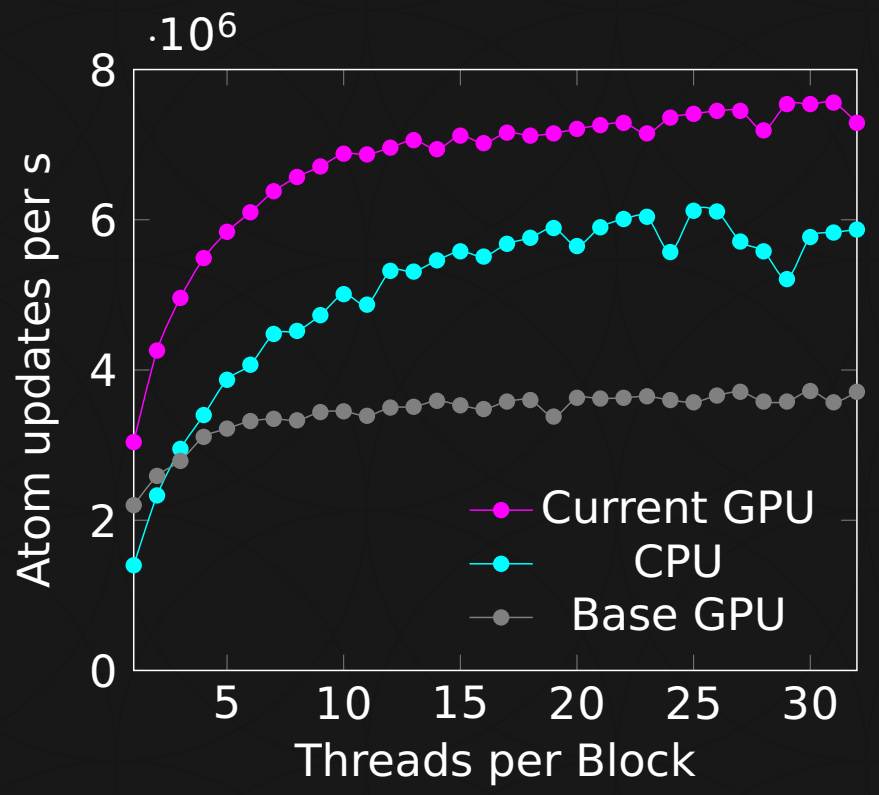
Force integration to AoS & CUDA II

- Performance



Current status: CPU vs GPU

- Performance



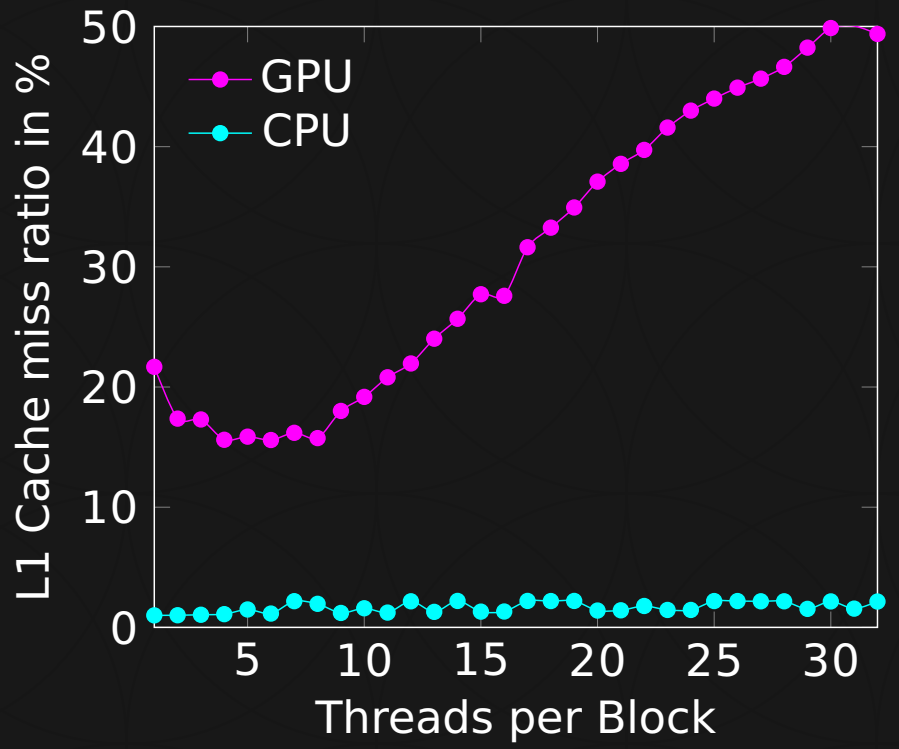
Kernel Analysis

Kernel Metrics

- *NVIDIA Nsight Compute* with full-profile is able to measure an extensive range of performance parameters, including
  - Cache hit ratio
  - Cache and memory bandwidth
  - FLOPS
- Similar metrics are evaluated for the CPU code using *likwid*

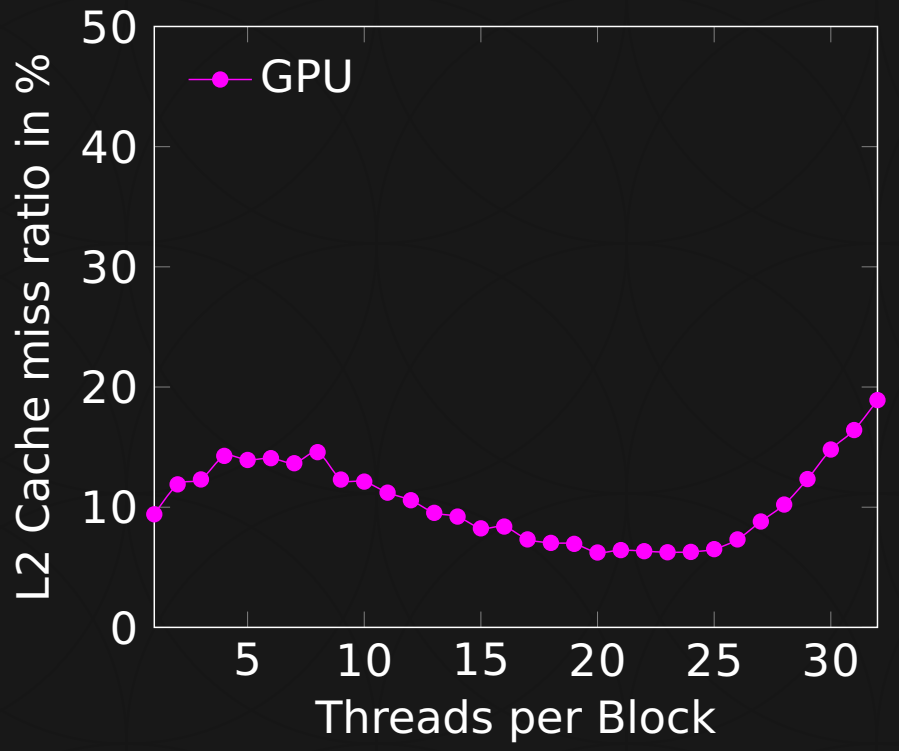
Kernel Metrics - L1 Cache

- L1 Cache miss ratio



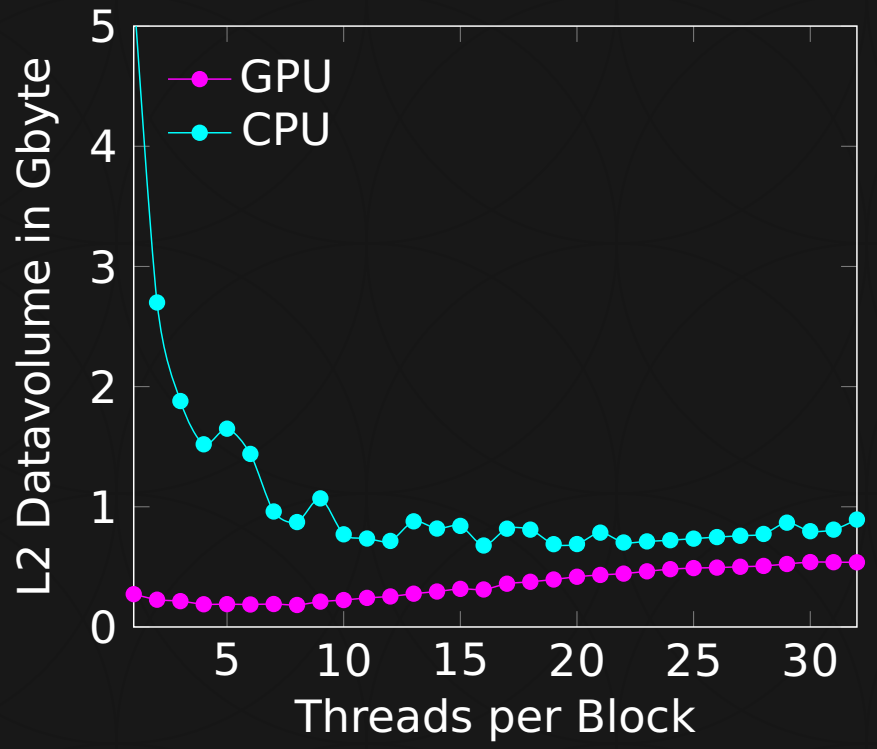
Kernel Metrics - L2 Cache I

- GPU L2 Cache miss ratio



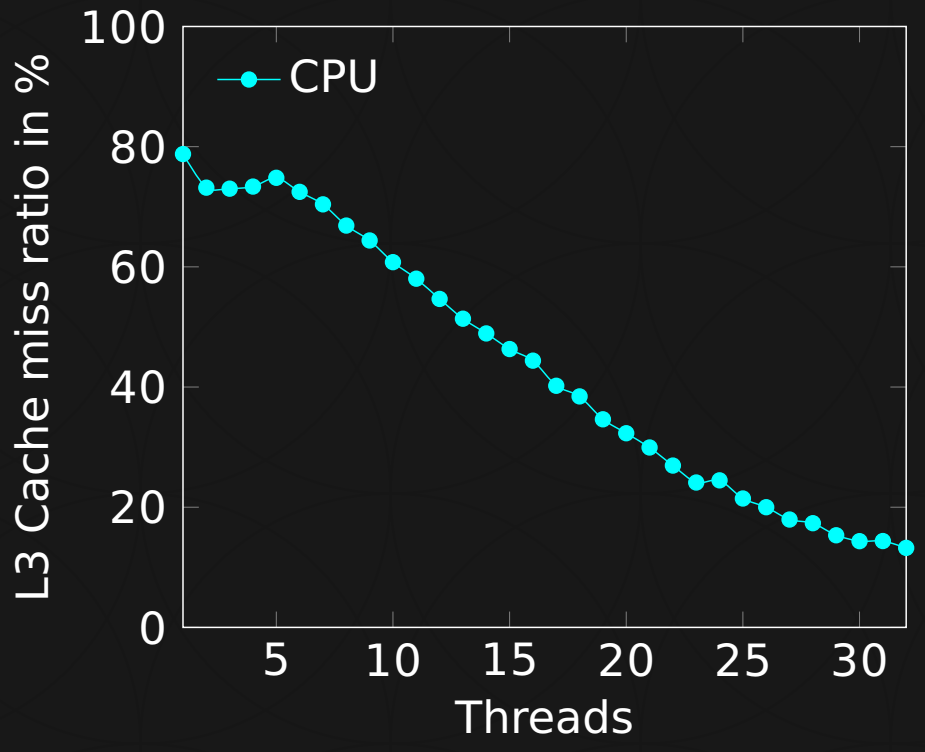
Kernel Metrics - L2 Cache II

- L2 Datavolume



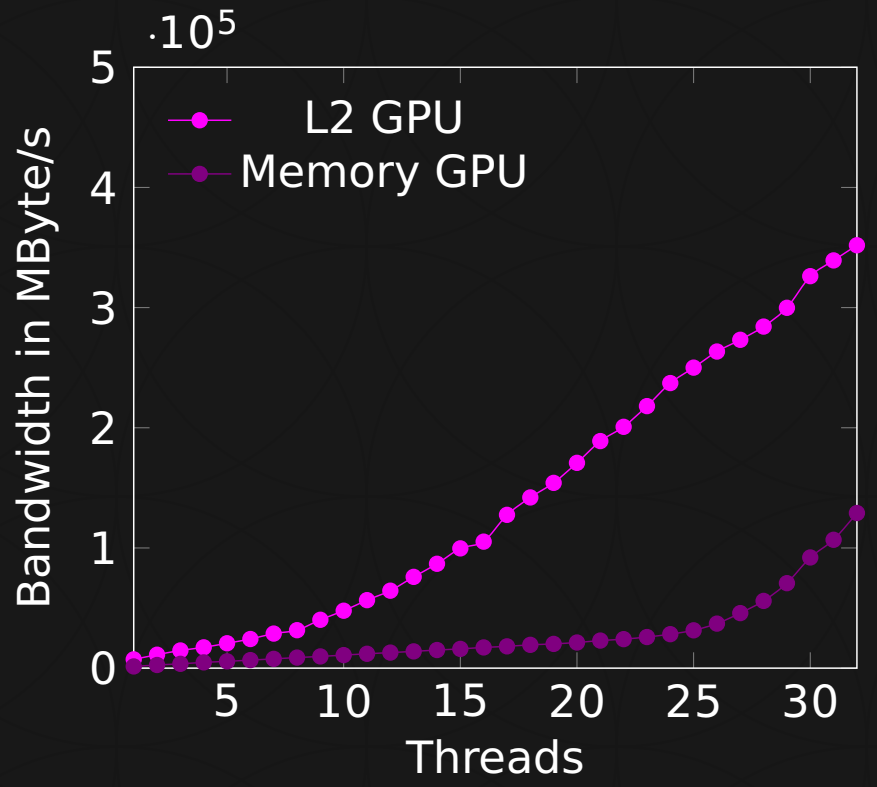
Kernel Metrics - L3 Cache

- CPU L3 Cache miss ratio



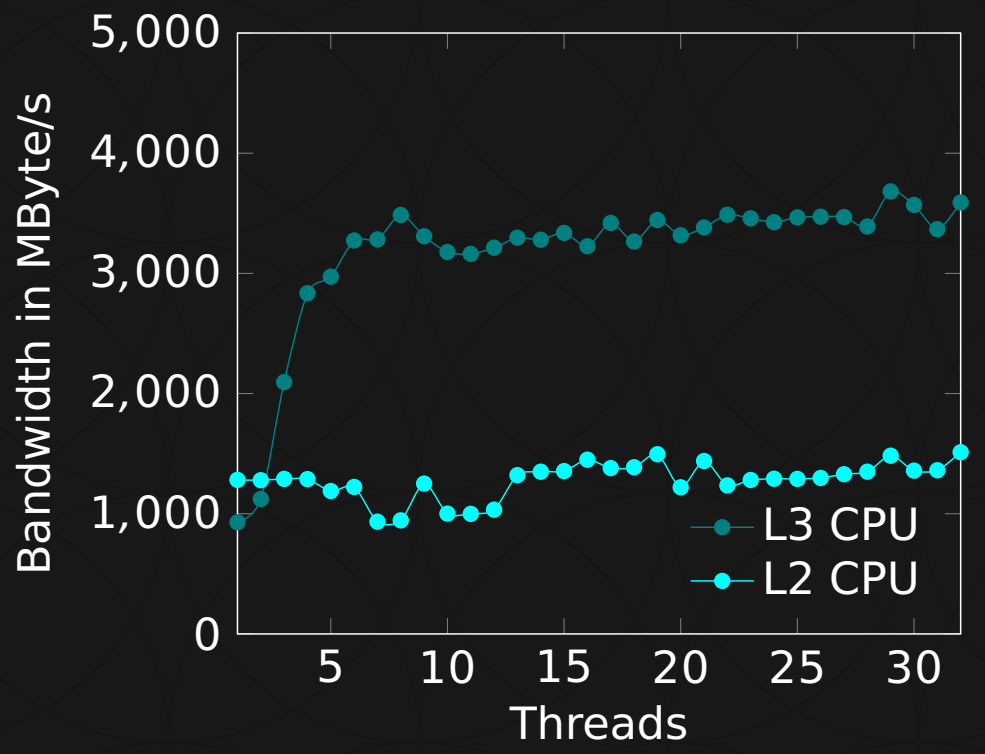
Kernel Metrics - GPU Bandwidth

- GPU L2 Cache / Memory Bandwidth



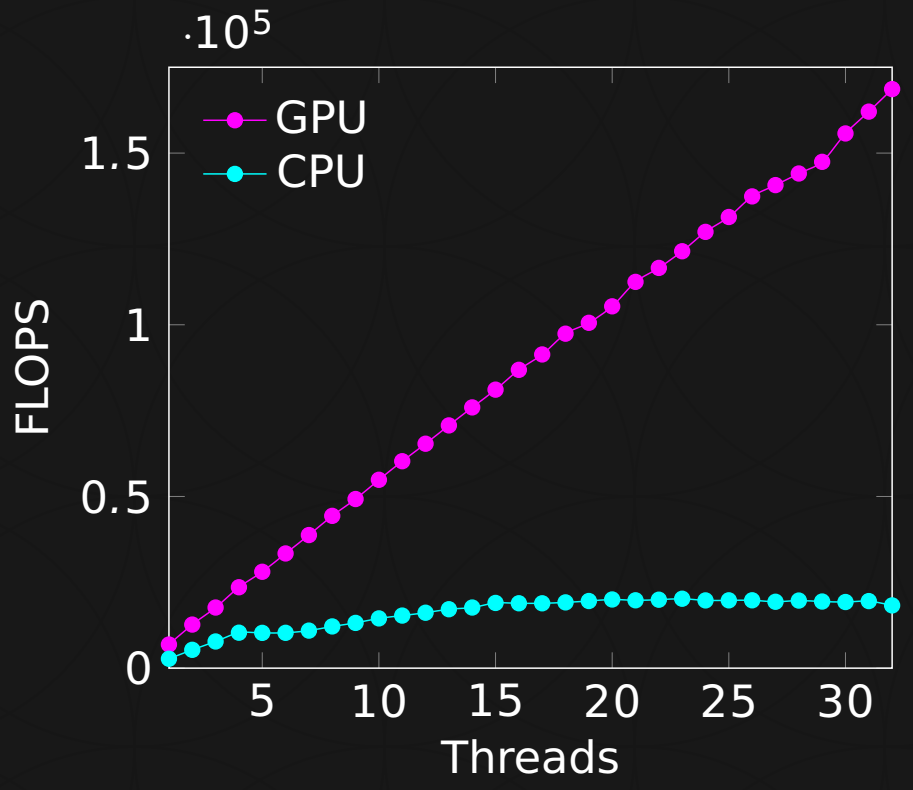
Kernel Metrics - CPU Bandwidth

- CPU L2/L3 Cache Bandwidth



Kernel Metrics - FLOPS

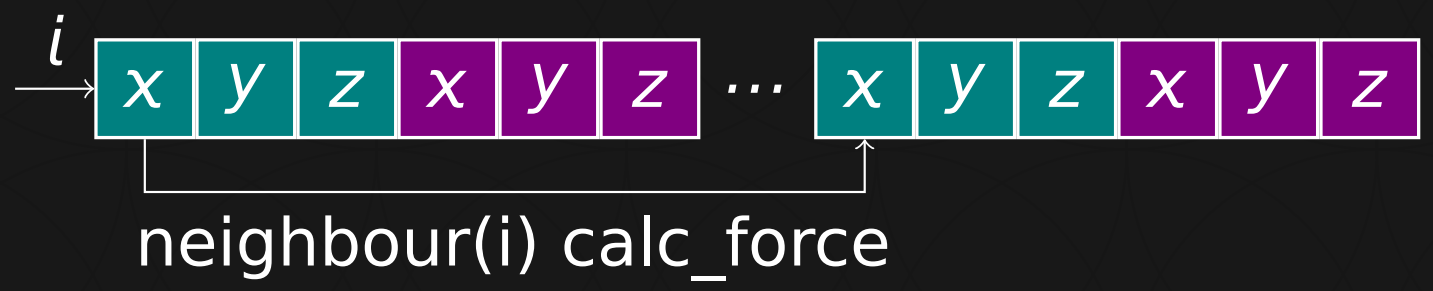
- Double precision FLOPS



Conclusions

Memory layout I

- Memory accesses to neighbouring atoms are not optimal:



- This affects cache miss ratio when multiple threads access the same data
  - Reorder such that neighbouring atoms are side by side and can be fetched within one cache line

Memory layout II

- NVIDIA Nsight Compute confirms:
  - Kernel accesses 7 bytes per thread per memory request but access pattern results in 768 bytes of L1 cache data transfer while $7 \cdot 32 = 224$ bytes would be optimal
    - Likely to occur because of distance between thread IDs ($\Rightarrow$ reordering)
  - Out of a L2 cache line of 128 bytes, only 38 bytes are used

Remaining parts

- Port reneighbour-function also to GPU such that no memory transfers to CPU within the main *for*-loop are necessary anymore

Bibliography I